

DATABASE MANAGEMENT SYSTEMS (IT 2040)

Question 1

What does JDBC stand for?

- A) Java Database Connection
- B) Java Database Connectivity
- C) Java Data Business Connector
- D) Java Database Communication

Answer: B

Question 2

What is the primary purpose of JDBC?

- A) To create databases
- B) To provide a common API for Java applications to access different databases
- C) To replace SQL
- D) To improve database performance

Answer: B

Question 3

In JDBC architecture, what is the role of the JDBC Driver Manager?

- A) Stores database data
- B) Manages database drivers and matches connection requests to appropriate drivers
- C) Executes SQL queries
- D) Creates database tables

Answer: B

Question 4

How many types of JDBC drivers are there?

- A) 2
- B) 3
- C) 4

- D) 5

Answer: C

Question 5

What is a Type 1 JDBC driver?

- A) Native API Driver
- B) JDBC-ODBC Bridge Driver
- C) Network Pure Java Driver
- D) Thin Driver

Answer: B

Question 6

Which JDBC driver type is written entirely in Java and provides the highest performance?

- A) Type 1
- B) Type 2
- C) Type 3
- D) Type 4

Answer: D

Question 7

What does a Type 2 (Native API) driver require on the client machine?

- A) Only Java Runtime Environment
- B) Vendor-specific native library/driver
- C) ODBC driver
- D) Middleware server

Answer: B

Question 8

Which driver type uses a three-tier approach with middleware?

- A) Type 1
- B) Type 2
- C) Type 3

- D) Type 4

Answer: C

Question 9

What is the correct order of steps to create a JDBC application?

- A) Open connection → Import packages → Load driver → Execute query
- B) Import packages → Load driver → Open connection → Execute query → Extract data → Clean up
- C) Load driver → Open connection → Import packages → Execute query
- D) Execute query → Open connection → Load driver

Answer: B

Question 10

Which package contains the JDBC API?

- A) java.jdbc.*
- B) java.sql.*
- C) java.database.*
- D) javax.sql.*

Answer: B

Question 11

How do you register a JDBC driver in your program?

- A) new Driver();
- B) Class.forName("driverClassName").newInstance();
- C) DriverManager.register("driverClassName");
- D) import driver.class;

Answer: B

Question 12

Which method is used to establish a database connection?

- A) Connection.getConnection()
- B) Database.connect()
- C) DriverManager.getConnection()

- D) `Driver.getConnection()`

Answer: C

Question 13

What are the three parameters required for `DriverManager.getConnection()`?

- A) Driver name, database name, port number
- B) URL, username, password
- C) Host, port, database name
- D) Connection string, timeout, retry count

Answer: B

Question 14

What does the JDBC URL format typically look like?

- A) `http://database:port/dbname`
- B) `jdbc:subprotocol:subname`
- C) `database://host:port`
- D) `sql://server/database`

Answer: B

Question 15

Which interface represents a database connection in JDBC?

- A) `DatabaseConnection`
- B) `Connection`
- C) `Statement`
- D) `Session`

Answer: B

Question 16

How many `Statement` interfaces are there in JDBC?

- A) 1
- B) 2
- C) 3

- D) 4

Answer: C

Question 17

When should you use a Statement object?

- A) For parameterized queries
- B) For calling stored procedures
- C) For static SQL statements executed once
- D) For batch processing

Answer: C

Question 18

When should you use a PreparedStatement instead of Statement?

- A) For DDL statements
- B) For SQL statements with parameters or executed multiple times
- C) Only for SELECT queries
- D) For database administration tasks

Answer: B

Question 19

What symbol represents a parameter placeholder in PreparedStatement?

- A) @
- B) \$
- C) ?
- D) #

Answer: C

Question 20

Which method is used to bind an integer value to a PreparedStatement parameter?

- A) bindInt()
- B) setInteger()
- C) setInt()
- D) putInt()

Answer: C

Question 21

Given: PreparedStatement pstmt = conn.prepareStatement("UPDATE Employee SET salary=? WHERE id=?"); How do you set salary to 50000 and id to 101?

- A) pstmt.setInt(50000, 1); pstmt.setInt(101, 2);
- B) pstmt.setFloat(1, 50000); pstmt.setInt(2, 101);
- C) pstmt.setInt(?, 50000); pstmt.setInt(?, 101);
- D) pstmt.setSalary(50000); pstmt.setld(101);

Answer: B

Question 22

Which interface is used to call stored procedures?

- A) Statement
- B) PreparedStatement
- C) CallableStatement
- D) ProcedureStatement

Answer: C

Question 23

How do you prepare a CallableStatement to execute a stored procedure?

- A) conn.createCallableStatement("procedureName");
- B) conn.prepareCall("{call procedureName(?, ?)}");

- C) conn.executeProc("procedureName");
- D) new CallableStatement("procedureName");

Answer: B

Question 24

How do you register an OUT parameter in CallableStatement?

- A) stmt.setOutParameter(index, type);
- B) stmt.registerOutParameter(index, type);
- C) stmt.declareOutput(index, type);
- D) stmt.outputParam(index, type);

Answer: B

Question 25

Which method executes a SELECT query and returns results?

- A) execute()
- B) executeUpdate()
- C) executeQuery()
- D) executeSelect()

Answer: C

Question 26

What does executeUpdate() return?

- A) ResultSet object
- B) Boolean value
- C) Number of rows affected

- D) Connection object

Answer: C

Question 27

What does the execute() method return?

- A) Number of rows
- B) ResultSet
- C) Boolean - true if ResultSet available, false otherwise
- D) Connection status

Answer: C

Question 28

What interface represents the result of a SELECT query?

- A) QueryResult
- B) ResultSet
- C) DataSet
- D) RecordSet

Answer: B

Question 29

How do you navigate to the next row in a ResultSet?

- A) rs.moveToNext()
- B) rs.next()
- C) rs.advance()
- D) rs.getNext()

Answer: B

Question 30

Where is the ResultSet cursor positioned initially?

- A) First row
- B) Last row
- C) Before the first row

- D) After the last row

Answer: C

Question 31

How can you retrieve data from a ResultSet column?

- A) By column index only
- B) By column name only
- C) By column index or column name
- D) Cannot retrieve data

Answer: C

Question 32

Given: `ResultSet rs = stmt.executeQuery("SELECT name, salary FROM Employee");` Which is correct to retrieve data?

- A) `String name = rs.getString(0); float salary = rs.getFloat(1);`
- B) `String name = rs.getString(1); float salary = rs.getFloat(2);`
- C) `String name = rs.getString("name"); float salary = rs.getFloat("salary");`
- D) Both B and C

Answer: D

Question 33

What exception is most commonly encountered in JDBC?

- A) `IOException`
- B) `SQLException`
- C) `DatabaseException`
- D) `JDBCException`

Answer: B

Question 34

Which `SQLException` method returns a vendor-specific error code?

- A) `getCode()`
- B) `getErrorCode()`
- C) `getVendorCode()`

- D) getErrorNumber()

Answer: B

Question 35

Why is it important to close database resources (Connection, Statement, ResultSet)?

- A) To improve query performance
- B) To prevent resource leaks and allow database to release resources
- C) To delete the data
- D) To commit transactions

Answer: B

SECTION B: MULTIPLE ANSWER QUESTIONS (Select ALL that apply) - 10 Questions

Question 36

Which are valid JDBC driver types? **(Select all that apply)**

- A) JDBC-ODBC Bridge Driver (Type 1)
- B) Native API Driver (Type 2)
- C) Middleware Driver (Type 3)
- D) Thin Driver (Type 4)
- E) Thick Driver (Type 5)
- F) Network Pure Java Driver (Type 3)

Answers: A, B, D, F

Question 37

Advantages of Type 4 (Thin) JDBC driver include: **(Select all that apply)**

- A) Written entirely in Java
- B) Platform independent
- C) Highest performance
- D) Requires ODBC installation
- E) Database vendor usually provides it
- F) No client-side software needed

Answers: A, B, C, E, F

Question 38

Which interfaces are part of java.sql package? **(Select all that apply)**

- A) Driver
- B) Connection
- C) Statement
- D) ResultSet
- E) DatabaseConnection
- F) PreparedStatement

Answers: A, B, C, D, F

Question 39

Which statement types can execute SQL queries? **(Select all that apply)**

- A) Statement
- B) PreparedStatement
- C) CallableStatement
- D) ResultSet
- E) Connection
- F) DriverManager

Answers: A, B, C

Question 40

PreparedStatement advantages over Statement include: **(Select all that apply)**

- A) Precompiled - better performance for repeated execution
- B) Prevents SQL injection attacks
- C) Supports parameterized queries
- D) Can call stored procedures
- E) More readable code with parameters
- F) Automatic data type conversion

Answers: A, B, C, E

Question 41

Which methods can execute SQL in Statement interface? **(Select all that apply)**

- A) execute()
- B) executeQuery()
- C) executeUpdate()
- D) executeSQL()
- E) executeBatch()
- F) query()

Answers: A, B, C, E

Question 42

For CallableStatement, which are TRUE? **(Select all that apply)**

- A) Extends PreparedStatement
- B) Used to call stored procedures
- C) Supports OUT parameters
- D) Uses {call procName(?)} syntax
- E) Can only call procedures, not functions
- F) Must register OUT parameters before execution

Answers: A, B, C, D, F

Question 43

ResultSet navigation and data retrieval methods include: **(Select all that apply)**

- A) next() - move to next row
- B) previous() - move to previous row (scrollable ResultSet)
- C) getInt() - retrieve integer value
- D) getString() - retrieve string value
- E) getFloat() - retrieve float value
- F) getRow() - get current row number

Answers: A, B, C, D, E, F

Question 44

Proper resource cleanup in JDBC includes: **(Select all that apply)**

- A) Closing ResultSet
- B) Closing Statement

- C) Closing Connection
- D) Must close in order: ResultSet → Statement → Connection
- E) Use try-with-resources (Java 7+)
- F) Closing Connection automatically closes Statement and ResultSet

Answers: A, B, C, D, E, F

Question 45

SQLException provides which information methods? **(Select all that apply)**

- A) getMessage() - error description
- B) getErrorCode() - vendor-specific error code
- C) getSQLState() - SQLState string
- D) getNextException() - chain of exceptions
- E) getCause() - underlying cause
- F) printStackTrace() - stack trace

Answers: A, B, C, D, E, F

SECTION C: CODE ANALYSIS & SCENARIO QUESTIONS - 5 Questions

Question 46

Analyze the following JDBC code:

```
Class.forName("com.mysql.jdbc.Driver");
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/myDB", "user", "pass");
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM Employee");
while(rs.next()) {
    System.out.println(rs.getString("name"));
}
```

What is missing from this code?

- A) Import statements
- B) Exception handling
- C) Resource cleanup (closing connections)
- D) All of the above

Answer: D

Question 47

Given the following stored procedure:

```
CREATE PROCEDURE getEmployeeDetails(  
    IN empId INT,  
    OUT empName VARCHAR(50),  
    OUT salary FLOAT  
)  
BEGIN  
    SELECT name, salary INTO empName, salary  
    FROM Employee WHERE id = empId;  
END
```

Which Java code correctly calls this procedure?

// Option A

```
CallableStatement cstmt = conn.prepareCall("{call getEmployeeDetails(?, ?, ?)}");  
cstmt.setInt(1, 101);  
cstmt.registerOutParameter(2, Types.VARCHAR);  
cstmt.registerOutParameter(3, Types.FLOAT);  
cstmt.execute();  
String name = cstmt.getString(2);  
float salary = cstmt.getFloat(3);
```

// Option B

```
CallableStatement cstmt = conn.prepareCall("exec getEmployeeDetails ?, ?, ?");  
cstmt.setInt(1, 101);  
cstmt.execute();  
String name = cstmt.getString(2);  
float salary = cstmt.getFloat(3);
```

Answer: A

Question 48

Security concern - SQL Injection:

// Version 1

```
String userId = request.getParameter("userId");
String sql = "SELECT * FROM Users WHERE id = " + userId + """;
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

// Version 2

```
String userId = request.getParameter("userId");
String sql = "SELECT * FROM Users WHERE id = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, userId);
ResultSet rs = pstmt.executeQuery();
```

Which version is secure and why?

- A) Version 1 - simpler and faster
- B) Version 2 - prevents SQL injection through parameterization
- C) Both are equally secure
- D) Neither is secure

Answer: B

Question 49

Performance Scenario:

You need to insert 10,000 employee records. Which approach is BEST?

// Approach A: Statement with loop

```
Statement stmt = conn.createStatement();
for(int i=0; i<10000; i++) {
    stmt.executeUpdate("INSERT INTO Employee VALUES(" + i + ", 'Name" + i + ")");
}
```

// Approach B: PreparedStatement with loop

```
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO Employee VALUES(?, ?)");
for(int i=0; i<10000; i++) {
```

```

    pstmt.setInt(1, i);
    pstmt.setString(2, "Name" + i);
    pstmt.executeUpdate();
}

```

// Approach C: PreparedStatement with batch

```

PreparedStatement pstmt = conn.prepareStatement("INSERT INTO Employee VALUES(?, ?)");
for(int i=0; i<10000; i++) {
    pstmt.setInt(1, i);
    pstmt.setString(2, "Name" + i);
    pstmt.addBatch();
}
pstmt.executeBatch();

```

Answer: C

Question 50

Complete the JDBC application with proper resource management:

```

Connection conn = null;

```

```

PreparedStatement pstmt = null;

```

```

ResultSet rs = null;

```

```

try {
    Class.forName("com.mysql.jdbc.Driver");
    conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/myDB", "user", "pass");

    pstmt = conn.prepareStatement("SELECT name, salary FROM Employee WHERE dept = ?");
    pstmt.setString(1, "IT");
    rs = pstmt.executeQuery();

    while(rs.next()) {
        System.out.println(rs.getString(1) + ": " + rs.getFloat(2));
    }
}

```



```

    }
}
catch(ClassNotFoundException e) {
    System.out.println("Driver not found: " + e.getMessage());
}
catch(SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
finally {
    // What goes here?
}

```

What should the finally block contain?

// Option A

```

finally {
    conn.close();
}

```

// Option B

```

finally {
    if(rs != null) rs.close();
    if(pstmt != null) pstmt.close();
    if(conn != null) conn.close();
}

```

// Option C

```

finally {
    try {
        if(rs != null) rs.close();
        if(pstmt != null) pstmt.close();
        if(conn != null) conn.close();
    }
    catch(SQLException e) {

```

```
        e.printStackTrace();  
    }  
}
```

Answer: C